

CMSIS-RTOS V2

Ejemplos

Version 1.0

Table of Contents

1. Ejemplos básicos CMSIS RTOS V2	5
• 1.1. Documentación en otros idiomas	
• 1.2. Descarga del código	
• 1.3. Listado de ejemplos incluidos	
• 1.4. Configuración del Proyecto de Keil	
• 1.4.1. Uso del simulador	
• 1.4.2. Uso del hardware	
• 1.4.3. Depuración de aplicaciones usando CMSIS-RTOS V2	
• 1.4.4. Symbols Window	
• 1.4.5. Logic Analyzer	
• 1.4.6. Performance Analyzer	
• 1.5. Código en main.c de los ejemplos	
• 1.5.1. Algunos detalles importantes	
2. Uso básico de un thread en CMSIS RTOS v2	18
• 2.1. Descripción General de ejemplothread	
• 2.2. Estructura de Datos	
• 2.3. Inicialización de los hilos	
• 2.4. Función del hilo	
• 2.5. Uso de HAL y CMSIS RTOS	
• 2.6. Código Fuente	
• 2.7. Dependencias	
• 2.8. Preguntas y respuestas sobre ejemplothread	
• 2.8.1. ¿Qué hace este código?	
• 2.8.2. ¿Qué es la estructura <i>mygpio_pin</i> ?	
• 2.8.3. ¿Cómo se inicializa el hilo?	
• 2.8.4. ¿Qué hace la función <i>Thread()</i> ?	
• 2.8.5. ¿Qué significa <i>osDelay()</i> ?	
• 2.8.6. ¿Qué pasa si <i>osThreadNew()</i> devuelve NULL?	
• 2.8.7. ¿Qué ficheros de cabecera se utilizan?	
• 2.8.8. Determine la carga de la CPU en esta aplicación	
3. Uso básico de threads en CMSIS RTOS v2	29
• 3.1. Descripción General	

- 3.2. Estructura `mygpio_pin`
- 3.3. Inicialización de los threads
- 3.4. `Thread()`
- 3.5. HAL y CMSIS RTOS
- 3.6. Código
- 3.7. Dependencias software
- 3.8. Preguntas y respuestas sobre `ejemplothreads`
 - 3.8.1. ¿Qué función hace este código?
 - 3.8.2. ¿Qué función tiene `mygpio_pin`?
 - 3.8.3. ¿Cómo se inicializan los hilos?
 - 3.8.4. ¿Qué función tiene `Thread()`?
 - 3.8.5. ¿Se ejecutan los hilos al mismo tiempo?
 - 3.8.6. ¿Qué función tiene `osDelay()`?
 - 3.8.7. ¿Qué pasa si `osThreadNew()` retorna NULL?
 - 3.8.8. ¿Qué includes se utilizan?
 - 3.8.9. ¿Cuanto vale el valor del tick es esta aplicación?
 - 3.8.10. ¿Que es el thread Idle? ¿Qué tamaño de stack tiene? ¿Y otro thread? ¿Que tamaño de stack usa?

4. Uso básico de threads y colas en CMSIS RTOS v2

41

- 4.1. Descripción General
- 4.2. Estructuras de Datos
- 4.3. Creación Hilos
- 4.4. Producer
- 4.5. Consumer
- 4.6. HAL-CMSIS RTOS
- 4.7. Fuente
- 4.8. Dependencias de software
- 4.9. Preguntas y respuestas sobre `ejemplothreads-queues`
 - 4.9.1. ¿Cuál es el propósito de la cola de mensajes `id_MsgQueue` en esta aplicación?
 - 4.9.2. ¿Qué función cumple el bucle anidado en el hilo `Producer`?
 - 4.9.3. ¿Cuanto tiempo tarda en llenarse la cola de mensajes `id_MsgQueue`?
 - 4.9.4. ¿cuanto vale la variable `errors_or_timeouts` despues de 1 minuto de ejecución del código?

5. Uso básico de threads y flags en CMSIS RTOS v2

53

- 5.1. Descripción
- 5.2. Función del hilo `Producer`

- 5.3. Función del hilo Consumer
- 5.4. HAL y CMSIS RTOS
- 5.5. Código aplicación
- 5.6. Dependencias de software del ejemplo
- 5.7. Preguntas y respuestas sobre ejemplothreads-flags
 - 5.7.1. Se modifica el código del Producer para que envíe ambas señales (0x0001 y 0x0002) de forma casi simultánea, seguido de un delay de 1 segundo:

6. Uso básico de threads y software timers en CMSIS RTOS v2

63

- 6.1. Descripción General
- 6.2. Función del Hilo **Timers**
- 6.3. Uso de HAL y CMSIS RTOS
- 6.4. Código específico
- 6.5. Dependencia
- 6.6. Preguntas y respuestas sobre ejemplothreads-timers
 - 6.6.1. ¿Cual es la diferencia fundamental entre un timer periódico otro one-shot?
 - 6.6.2. Los ficheros RTX_config.h y RTX_config.c son generados automáticamente por el entorno de desarrollo. ¿Se pueden modificar?
 - 6.6.3. Si se fija un punto de ruptura en la línea 47, ¿qué se espera ver en el **Watch Windows**->**RTX RTOS** ?

7. Requisitos previos

71

- 7.1. Software que se instalará
- 7.2. Hardware necesario

8. Instalación de Visual Studio Code

73

- 8.1. Descarga
- 8.2. Instalación
- 8.3. Verificación
- 8.4. Configuración recomendada del editor

1. Ejemplos básicos CMSIS RTOS V2

El repositorio contiene ejemplos básicos para entender el funcionamiento de la API CMSIS-RTOS V2 utilizando el sistema operativo RTX version 5. Los ejemplos están implementados para el STM32F429 y utilizan mínimamente los periféricos del microcontrolador y hacer hincapié en los conceptos de manejo del Sistema Operativo.

Note

Uso de los ejemplos

Los ejemplos se han implementado para la asignatura **Sistemas Basados en Microprocesador** de la **(ETSI Sistemas de Telecomunicación) Universidad Politécnica de Madrid** y se pueden ejecutar utilizando el simulador del microprocesador incluido en el entorno de ARM keil Microvision o bien el hardware.

1.1. Documentación en otros idiomas

Traducción a inglés - English translation

1.2. Descarga del código

Para descargar el código puede utilizar un cliente de git en su ordenador o bien descargar el repositorio completo (formato **zip**). Las instrucciones para clonar el repositorio son:

Note

Descarga del código

```
$ git clone https://github.com/mruizglz/SBM-rtos.git
```

1.3. Listado de ejemplos incluidos

Table 1.1 Ejemplos incluidos

Carpeta	Objetivos
ejemplothreads	Aprender el manejo básico de creación de threads. Uso de la misma función con parámetros para crear múltiples threads
ejemplothreads-flags	Sincronización de threads usando flags
ejemplothreads-queues	Intercambio de datos entre threads usando colas
ejemplothreads-timers	Gestión de timers “software”

1.4. Configuración del Proyecto de Keil

1.4.1. Uso del simulador

ARM Keil Microvision dispone de opciones para configurar donde se ejecutará la aplicación (Icono *Options for Target*). Seleccione **Debug** y active el uso del simulador (**Use Simulator**). Es necesario que configure el fichero de inicialización (**Initialization File**) para que cargue un script de configuración del microcontrolador. En este caso, seleccione el fichero `simulator.ini` que se encuentra en cada una de las carpetas de ejemplo. Por ejemplo en la carpeta `.\ejemplothreads` del repositorio encontrará el fichero **simulador.ini** con este contenido:

```
1 MAP 0x40000000,0x400FFFFF read write
2 MAP 0xE0000000,0xE00FFFFF read write
```

El significado de estas instrucciones es habilitar para el simulador las operaciones de lectura/escritura en las zonas de memoria donde se encuentran los periféricos.

Tip

Cuando se usa el simulador de Keil las operaciones de escritura y lectura de los periféricos no tienen ningún efecto y por tanto no podrá simular el comportamiento hardware de los mismos. Todas las operaciones de la capa HAL que actúan sobre periféricos no tendrán ningún efecto. Por ejemplo, si en el código se configura un pin como salida y luego se escribe un valor alto en el mismo, no podrá ver ningún cambio en el estado del pin.

Como podrá ver en el código del programa `main.c` existe compilación condicional para incluir o no el código de configuración del RCC para usar un reloj externo (HSE). Si utiliza el simulador debe desactivar esta opción y usar el reloj interno (HSI) que es el que utiliza el simulador. La pestaña **C/C++(AC6)** permite añadir en `define` etiquetas. Incluya `SIMULATOR` si quiere utilizar el simulador.

1.4.2. Uso del hardware

Si dispone de una placa con el microcontrolador STM32F429 puede ejecutar el código directamente en el hardware. En este caso debe configurar las opciones del proyecto para que utilice el ST-Link en lugar del simulador.

No defina la variable `SIMULATOR` en las opciones de compilación para que el circuito de RCC se configure adecuadamente.

1.4.3. Depuración de aplicaciones usando CMSIS-RTOS V2

La depuración de las aplicaciones se debe realizar combinando el uso de puntos de ruptura y de la aplicación `RTX RTOS view` disponible en el menú `View->Watch Windows->RTX RTOS`. Esta permite ver el estado en el que se encuentran los diferentes objetos del sistema operativo cuando el procesador pausa su ejecución. Herramientas complementarias para entender el funcionamiento de una aplicación son: `Logyc Analyzer`, `Performance Analyzer`, `System Analyzer`, `Event Recorder`, `Event Statistics` y `Symbols Window`.

1.4.4. Symbols Window

La opción **Symbols Window** permite visualizar y explorar todos los símbolos definidos en el proyecto, incluyendo variables globales, variables estáticas, funciones y direcciones de registros. Esta ventana es útil para depuración y análisis en tiempo real.

- Muestra una lista jerárquica de todos los símbolos disponibles en el programa cargado.
- Permite buscar y filtrar símbolos por nombre.
- Muestra la dirección y el valor actual de cada símbolo durante la sesión de depuración.
- Facilita el arrastre de variables a otras ventanas de análisis, como el Watch Window o el Logic Analyzer.
- Permite examinar variables optimizadas si están disponibles en la tabla de símbolos.
- Si un símbolo no aparece, verifique la configuración de optimización del compilador y el ámbito de la variable.

Para utilizarlo:

1. Iniciar una sesión de depuración.

2. Abrir la ventana desde el menú: View ► Symbol Window.
3. Buscar el símbolo deseado utilizando el campo de filtro.
4. Arrastrar el símbolo a la ventana de Watch o **Logic Analyzer** para su monitorización.

1.4.5. Logic Analyzer

Permite visualizar la evolución temporal de variables que sean globales a la aplicación, el contenido de posiciones de memoria, etc. Se puede configurar el rango de valores y es muy apropiado para comparar visualmente la evolución de la aplicación software a través del seguimiento de variables. Para agregar señales al **Logic Analyzer** puede arrastrarlas de la ventana de símbolos o escribir el nombre de la misma.

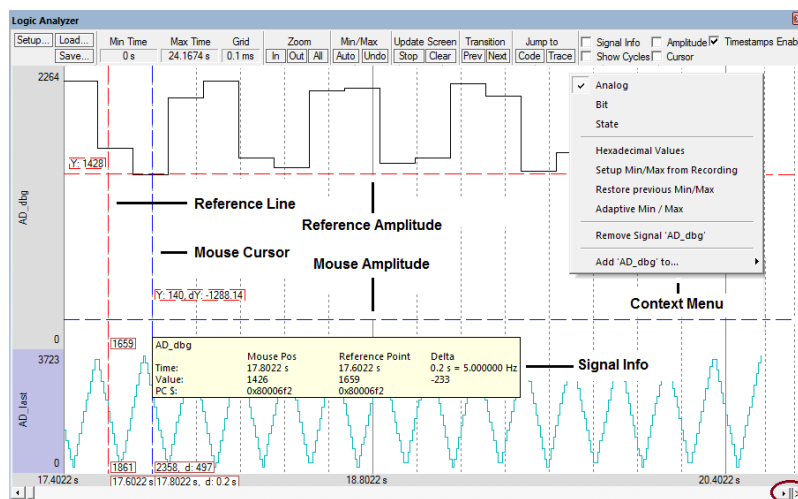


Fig. 1.1 Analizador lógico de ARM Keil Microvision.

1.4.6. Performance Analyzer

Warning

Solo esta disponible en Simulación porque el ST-LINK no lo soporta

Permite conocer el porcentaje de tiempo utilizado por cada porción del código de nuestra aplicación. Para utilizarlo:

1. Iniciar una sesión de depuración.
2. Abrir la ventana desde el menú: View ► Analysis Windows ► Performance analyzer.

3. Se muestra una lista de las diferentes secciones de código.

Module/Function	Calls	Time(Sec)	Time(%)
threads		5.196 s	100%
RTE/CMSIS/RTX_Config.c	1	5.117 s	98%
osRtxIdleThread	0	0us	0%
osRtxErrorNotify		28.590 ms	1%
./././Source/rtx_thread.c		19.020 ms	0%
./././Source/rtx_system.c		14.929 ms	0%
./././Source/GCC/irq_armv7m.S		7.468 ms	0%
./././Source/rtx_timer.c		2.277 ms	0%
./././Source/os_systick.c		2.112 ms	0%
./././Source/rtx_delay.c		1.906 ms	0%
main.c		1.715 ms	0%
C:/Users/Usuario/AppData/Local/Am/Packs/Keil/STM32F4xx_DFP/2.15.0/Drv...		1.423 ms	0%
Thread.c		29.833 us	0%
./././Source/rtx_memory.c		28.417 us	0%
./././Source/rtx_kernel.c		20.583 us	0%
./././Source/rtx_msgqueue.c		5.417 us	0%
./././Source/rtx_mempool.c		5.333 us	0%
RTE/Device/STM32F429ZITx/system_stm32f4xx.c		4.583 us	0%
C:/Users/Usuario/AppData/Local/Am/Packs/Keil/STM32F4xx_DFP/2.15.0/Drv...		2.917 us	0%
C:/Users/Usuario/AppData/Local/Am/Packs/ARM/CMSIS/5.8.0/CMSIS/RTOS...		0.917 us	0%

1.5. Código en main.c de los ejemplos

El código del programa principal está detallado a continuación y es el mismo o muy similar para todos los ejemplos:

```

1 #include "main.h"
2
3 #ifdef _RTE_
4 #include "RTE_Components.h"           // Component
selection
5 #endif
6 #ifdef RTE_CMSIS_RTOS2                 // when RTE
component CMSIS RTOS2 is used
7 #include "cmsis_os2.h"               // ::CMSIS:RTOS2
8 #endif
9
10 #ifdef RTE_CMSIS_RTOS2_RTX5
11 /**
12  * Override default HAL_GetTick function
13  */
14 uint32_t HAL_GetTick (void) {
15     static uint32_t ticks = 0U;
16     uint32_t i;
17
18     if (osKernelGetState () == osKernelRunning) {
19         return ((uint32_t)osKernelGetTickCount ());
20     }
21
22     /* If Kernel is not running wait approximately 1 ms
then increment
23     and return auxiliary tick counter value */
24     for (i = (SystemCoreClock >> 14U); i > 0U; i--) {
25         __NOP(); __NOP(); __NOP(); __NOP(); __NOP();
__NOP();
26         __NOP(); __NOP(); __NOP(); __NOP(); __NOP();
__NOP();
27     }
28     return ++ticks;
29 }
30
31 /**
32  * Override default HAL_InitTick function
33  */
34 HAL_StatusTypeDef HAL_InitTick(uint32_t TickPriority) {
35
36     UNUSED(TickPriority);

```

```

37
38     return HAL_OK;
39 }
40 #endif
41 #include "Timer.h"
42 /** @addtogroup STM32F4xx_HAL_Examples
43     * @{
44     */
45
46 /** @addtogroup Templates
47     * @{
48     */
49
50 /* Private typedef
-----
*/
51 /* Private define
-----
*/
52 /* Private macro
-----
-*/
53 /* Private variables
-----*/
54 /* Private function prototypes
-----*/
55 static void SystemClock_Config(void);
56 static void Error_Handler(void);
57
58 /* Private functions
-----*/
59 /**
60     * @brief Main program
61     * @param None
62     * @retval None
63     */
64
65 int main(void)
66 {
67
68     /* STM32F4xx HAL library initialization:
69         - Configure the Flash prefetch, Flash preread and
Buffer caches
70
71         - SysTick timer is configured by default as source
of time base, but user
72         can eventually implement his proper time
base source (a general purpose
73         timer for example or other time source),
keeping in mind that Time base
74         duration should be kept 1ms since
PPP_TIMEOUT_VALUES are defined and

```

```

74         handled in milliseconds basis.
75     - Low Level Initialization
76     */
77     HAL_Init();
78
79     /* Configure the system clock to 168 MHz */
80     SystemClock_Config();
81     SystemCoreClockUpdate();
82
83     /* Add your application code here
84     */
85
86 #ifndef RTE_CMSIS_RTOS2
87     /* Initialize CMSIS-RTOS2 */
88     osKernelInitialize ();
89
90     /* Create thread functions that start executing,
91     Example: osThreadNew(app_main, NULL, NULL); */
92     Init_Threads();
93     /* Start thread execution */
94     osKernelStart();
95 #endif
96
97     /* Infinite loop */
98     while (1)
99     {
100     }
101 }
102
103 /**
104  * @brief System Clock Configuration
105  * The system Clock is configured as follow :
106  * System Clock source = PLL
107  * (HSE)
108  * SYSCLK(Hz) =
109  * 168000000
110  * HCLK(Hz) =
111  * 168000000
112  * AHB Prescaler = 1
113  * APB1 Prescaler = 4
114  * APB2 Prescaler = 2
115  * HSE Frequency(Hz) = 8000000
116  * PLL_M = 25
117  * PLL_N = 336
118  * PLL_P = 2
119  * PLL_Q = 7
120  * VDD(V) = 3.3
121  * Main regulator output voltage = Scale1
122  * mode
123  * Flash Latency(WS) = 5
124  * @param None
125  * @retval None

```

```

122  */
123  static void SystemClock_Config(void)
124  {
125      RCC_ClkInitTypeDef RCC_ClkInitStruct;
126      RCC_OscInitTypeDef RCC_OscInitStruct;
127
128      /* Enable Power Control clock */
129      __HAL_RCC_PWR_CLK_ENABLE();
130
131      /* The voltage scaling allows optimizing the power
consumption when the device is
132      clocked below the maximum system frequency, to
update the voltage scaling value
133      regarding system frequency refer to product
datasheet. */
134
135      __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
136
137      /* Enable HSE Oscillator and activate PLL with HSE as
source */
138      RCC_OscInitStruct.OscillatorType =
RCC_OSCILLATORTYPE_HSE;
139      RCC_OscInitStruct.HSEState = RCC_HSE_ON;
140      RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
141      RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
142      RCC_OscInitStruct.PLL.PLLM = 4;
143      RCC_OscInitStruct.PLL.PLLN = 168;
144      RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
145      RCC_OscInitStruct.PLL.PLLQ = 7;
146      if(HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
147      {
148          /* Initialization Error */
149          Error_Handler();
150      }
151
152      /* Select PLL as system clock source and configure the
HCLK, PCLK1 and PCLK2
clocks dividers */
153      RCC_ClkInitStruct.ClockType = (RCC_CLOCKTYPE_SYSCLK |
RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_PCLK1 |
RCC_CLOCKTYPE_PCLK2);
154      RCC_ClkInitStruct.SYSCLKSource =
RCC_SYSCLKSOURCE_PLLCLK;
155      RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
156      RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
157      RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
158      if(HAL_RCC_ClockConfig(&RCC_ClkInitStruct,
FLASH_LATENCY_5) != HAL_OK)
159      {
160          /* Initialization Error */
161          Error_Handler();
162      }

```



```
163
164  /* STM32F405x/407x/415x/417x Revision Z devices:
prefetch is supported */
165  if (HAL_GetREVID() == 0x1001)
166  {
167      /* Enable the Flash prefetch */
168      __HAL_FLASH_PREFETCH_BUFFER_ENABLE();
169  }
170 }
171
172 /**
173  * @brief This function is executed in case of error
occurrence.
174  * @param None
175  * @retval None
176  */
177 static void Error_Handler(void)
178 {
179     /* User may add here some code to deal with this error
*/
180     while(1)
181     {
182     }
183 }
```

1.5.1. Algunos detalles importantes

1. Al utilizar el sistema operativo CMSIS-RTOS V2 se ha definido RTE_CMSIS_RTOS2_RTX5. Esto supone incluir dos funciones que anulan las definidas anteriormente. Estas funciones son `HAL_GetTick` y `HAL_InitTick`. La primera tiene dos comportamientos diferentes. Si el sistema operativo esta en ejecución devuelve el valor retornado por `osKernelGetTickCount` que indica el número de ticks que han pasado desde que se arranco el SO. * Si **no** se ha arrancado el SO la función produce un retardo de aproximadamente 1ms e incrementa la variable estática `ticks`. `Hal_GetTick` se utiliza por las librerías HAL de STM para controlar timeouts en la gestión de los periféricos. La segunda función, `HAL_InitTick`, es usada por la capa HAL para programar un timer HW que proporcione una interrupción cada 1ms. Por defecto este timer es el SysTick timer. Cuando no se usa el sistema operativo esta función realiza las operaciones del código descrito en `stm32f4xx_hal.c` Al usar el SO esta función se substituye por la definida en el `main.c`, que no realiza ninguna

operación. Es el código del sistema operativo quien se encarga de inicializar el SysTick (os_systick.c).

1. Después del código de inicialización de la librería HAL y de la configuración del RCC del micro (que por cierto utiliza la función HAL_GetTick) se procede a ejecutar la siguiente porción de código:

```
1 #ifdef RTE_CMSIS_RTOS2
2  /* Initialize CMSIS-RTOS2 */
3  osKernelInitialize ();
4
5  /* Create thread functions that start executing,
6   Example: osThreadNew(app_main, NULL, NULL); */
7  Init_Threads();
8  /* Start thread execution */
9  osKernelStart();
10 #endif
11
12 /* Infinite loop */
13 while (1)
14 {
15 }
```

La función `osKernelInitialize` inicializa el Sistema Operativo. La función `init_Threads` contiene el código de creación de los recursos necesarios en este ejemplo, y `osKernelStart` comienza la ejecución de los diferentes objetos del SO sin retornar. Eso quiere decir que la porción del código while es código muerto.

1. Toda la inicialización del hardware se debe hacer antes de lanzar la ejecución del SO. Puede incluirse en el código específico de cada thread, en funciones que se ejecuten entre `osKernelInitialize` y `osKernelStart`, o en funciones que se incluyan antes de llamar a `osKernelInitialize`. En cualquier caso debe tener cuidado con el mecanismo de retardo que utiliza en cada parte de la aplicación.

Note

No se recomienda el uso de la función HAL_Delay cuando se esta ejecutando el SO porque la función se puede quedar bloqueada.

2. Uso básico de un thread en CMSIS RTOS v2

Esta sección describe el funcionamiento de un programa (**ejemplothread**) en C que utiliza CMSIS RTOS v2 y la biblioteca HAL de STM32 para controlar un LED mediante un hilo.

2.1. Descripción General de ejemplothread

El programa crea un hilo que maneja un LED conectado al pin PB0 del microcontrolador STM32F4. El hilo alterna el estado del LED con una frecuencia configurable, utilizando funciones del sistema operativo en tiempo real (RTOS) y la biblioteca HAL para la configuración y manipulación de los pines GPIO.

2.2. Estructura de Datos

Se define una estructura llamada `mygpio_pin` que encapsula toda la información necesaria para controlar un LED:

- `GPIO_InitTypeDef pin`: configuración del pin (modo, velocidad, tipo de salida).
- `GPIO_TypeDef *port`: puerto GPIO al que pertenece el pin.
- `int delay`: retardo en ms entre cada cambio de estado del LED.
- `uint8_t counter`: contador que se alterna en cada iteración del hilo.

Esta estructura permite pasar todos los parámetros necesarios a la función del hilo de forma organizada.

2.3. Inicialización de los hilos

La función `Init_Thread` realiza las siguientes tareas:

1. Habilita el reloj del puerto GPIOB.
2. Configura `mygpio_pin` para el pin PB0.
3. Crea un hilo con `osThreadNew`, que ejecuta la función `Thread` pasándole una estructura `mygpio_pin` para el pin PB0.

2.4. Función del hilo

La función `Thread` realiza lo siguiente:

1. Inicializa el pin GPIO usando `HAL_GPIO_Init`.
2. Entra en un bucle infinito donde: - Alterna el valor del contador con `~counter`. - Cambia el estado del pin con `HAL_GPIO_TogglePin`. - Espera el tiempo definido en `delay` usando `osDelay`.

Esto provoca que el LED conectado al pin correspondiente parpadee con una frecuencia que es configurable.

2.5. Uso de HAL y CMSIS RTOS

- **HAL (Hardware Abstraction Layer):** se utiliza para configurar e inicializar los pines GPIO de forma sencilla y portable.
- **CMSIS RTOS v2:** proporciona las funciones para crear y gestionar hilos, como `osThreadNew` y `osDelay`.

2.6. Código Fuente

```
#include "cmsis_os2.h"
#include "stm32f4xx_hal.h"
#include <stdlib.h>

osThreadId_t tid_Thread;

GPIO_InitTypeDef led_ld1 = {
    .Pin = GPIO_PIN_0,
    .Mode = GPIO_MODE_OUTPUT_PP,
    .Pull = GPIO_NOPULL,
    .Speed = GPIO_SPEED_FREQ_LOW
};

typedef struct {
    GPIO_InitTypeDef pin;
    GPIO_TypeDef *port;
    int delay;
    uint8_t counter;
} mygpio_pin;

mygpio_pin pinB0;

int Init_Thread(void) {
    __HAL_RCC_GPIOB_CLK_ENABLE();

    pinB0.pin = led_ld1;
    pinB0.port = GPIOB;
    pinB0.delay = 15;
    pinB0.counter = 1;
    tid_Thread = osThreadNew(Thread, (void *)&pinB0, NULL);
    if (tid_Thread == NULL) return -1;

    return 0;
}

void Thread(void *argument) {
    mygpio_pin *gpio = (mygpio_pin *)argument;
    HAL_GPIO_Init(gpio->port, &(gpio->pin));
    while (1) {
        gpio->counter++;
        HAL_GPIO_TogglePin(gpio->port, gpio->pin.Pin);
        osDelay(gpio->delay);
    }
}
```

```
}  
}
```

2.7. Dependencias

- Librería HAL de STM32.
- CMSIS RTOS v2.

2.8. Preguntas y respuestas sobre ejemplothread

Esta sección contiene una serie de preguntas con sus respectivas respuestas sobre el funcionamiento del código que utiliza CMSIS RTOS v2 para controlar LEDs en una placa STM32.

2.8.1. ¿Qué hace este código?

Este código crea un hilo (thread) que controla un LED conectado al pin PB0 de una placa STM32F429. El hilo alterna el estado del LED (encendido/apagado) con una frecuencia determinada utilizando funciones del sistema operativo en tiempo real CMSIS RTOS v2. Dentro del código del Thread se realiza un casting al tipo de estructura que se utiliza en el ejemplo

2.8.2. ¿Qué es la estructura *mygpio_pin*?

Es una estructura de datos que encapsula la información necesaria para controlar un pin GPIO en este ejemplo:

- `pin`: configuración del pin (tipo, velocidad, modo).
- `port`: puerto GPIO al que pertenece el pin (por ejemplo, GPIOB).
- `delay`: retardo en ms entre cada cambio de estado (toggle).
- `counter`: variable auxiliar que cuenta la cantidad de veces que se ha realizado el toggle.

2.8.3. ¿Cómo se inicializa el hilo?

La función `Init_Thread()` habilita el reloj del puerto GPIOB, rellena los parámetros de la estructura y crea un hilo con la función `osThreadNew()`, pasando como argumento la estructura `mygpio_pin` correspondiente a cada LED.

2.8.4. ¿Qué hace la función *Thread()*?

La función `Thread(void *argument)` se encarga de:

1. Inicializar el pin GPIO usando `HAL_GPIO_Init`.
2. Ejecutar un bucle infinito donde: - Se incrementa el valor de `counter`. - Se cambia el estado del LED con `HAL_GPIO_TogglePin`. - Se espera el tiempo definido en `delay` usando `osDelay`.

2.8.5. ¿Qué significa *osDelay()*?

Es una función del RTOS que suspende la ejecución del hilo actual durante un número determinado de ms. Esto permite que otros hilos se ejecuten mientras tanto. `osDelay` tiene como parámetro el número de ticks que la tarea estará bloqueada. El número de ticks por segundo se define en el archivo `RTX_Config.h` (parámetro `Kernel Tick Frequency [Hz]`). En este ejemplo se ha configurado a 1000, por lo que un tick equivale a 1 ms.

2.8.6. ¿Qué pasa si *osThreadNew()* devuelve NULL?

Significa que no se pudo crear el hilo. En ese caso, la función `Init_Thread()` devuelve -1 como señal de error. Si el programa principal que llama a esta función no comprueba el retorno no hay ningún control de errores.

2.8.7. ¿Qué ficheros de cabecera se utilizan?

- `cmsis_os2.h`: para funciones del sistema operativo en tiempo real.
- `stm32f4xx_hal.h`: para funciones de acceso a hardware (HAL).
- `stdlib.h`: para funciones estándar de C que en este caso no se están incluyendo en el código.

2.8.8. Determine la carga de la CPU en esta aplicación

Para determinar la carga que supone la ejecución del thread para la CPU se puede utilizar la utilidad de `Performance Analyzer` en modo simulación. La carga de CPU obtenida es insignificante. Si se cambia en la estructura de datos el campo `delay` por 0 la carga del Thread pasa a ser del 19%.

3. Uso básico de threads en CMSIS RTOS v2

Esta sección describe el funcionamiento de un programa en C que utiliza CMSIS RTOS v2 y la biblioteca HAL de STM32 para controlar dos LEDs mediante hilos concurrentes.

3.1. Descripción General

El programa crea dos hilos que controlan dos LEDs conectados a los pines PB0 y PB7 del microcontrolador STM32F429. Cada hilo alterna el estado de su LED con una frecuencia distinta, utilizando funciones del sistema operativo en tiempo real (RTOS) y la biblioteca HAL para la configuración y manipulación de los pines GPIO.

3.2. Estructura mygpio_pin

Se define una estructura llamada `mygpio_pin` que encapsula toda la información necesaria para controlar un LED:

- `GPIO_InitTypeDef pin`: configuración del pin (modo, velocidad, tipo de salida).
- `GPIO_TypeDef *port`: puerto GPIO al que pertenece el pin.
- `int delay`: retardo en ms entre cada cambio de estado del LED.
- `uint8_t counter`: contador que se incrementa en cada iteración del hilo.

Esta estructura permite pasar todos los parámetros necesarios a la función del hilo de forma organizada.

3.3. Inicialización de los threads

La función `Init_Thread` realiza las siguientes tareas:

1. Habilita el reloj del puerto GPIOB.
2. Configura dos instancias de `mygpio_pin` para los pines PB0 y PB7.
3. Crea dos hilos con `osThreadNew`, cada uno ejecutando la función `Thread` con una instancia diferente de `mygpio_pin`.

Cada hilo se ejecuta de forma independiente y controla su propio LED.

3.4. Thread()

La función `Thread` realiza lo siguiente:

1. Inicializa el pin GPIO usando `HAL_GPIO_Init`.
2. Entra en un bucle infinito donde: - Alterna el valor del contador con `~counter`. - Cambia el estado del pin con `HAL_GPIO_TogglePin`. - Espera el tiempo definido en `delay` usando `osDelay`.

Esto provoca que el LED conectado al pin correspondiente parpadee con una frecuencia determinada.

3.5. HAL y CMSIS RTOS

- **HAL (Hardware Abstraction Layer):** se utiliza para configurar e inicializar los pines GPIO de forma sencilla y portable.
- **CMSIS RTOS v2:** proporciona las funciones para crear y gestionar hilos, como `osThreadNew` y `osDelay`.

3.6. Código

```
#include "cmsis_os2.h"
#include "stm32f4xx_hal.h"
#include <stdlib.h>

osThreadId_t tid_Thread;

GPIO_InitTypeDef led_ld1 = {
    .Pin = GPIO_PIN_0,
    .Mode = GPIO_MODE_OUTPUT_PP,
    .Pull = GPIO_NOPULL,
    .Speed = GPIO_SPEED_FREQ_LOW
};

GPIO_InitTypeDef led_ld2 = {
    .Pin = GPIO_PIN_7,
    .Mode = GPIO_MODE_OUTPUT_PP,
    .Pull = GPIO_NOPULL,
    .Speed = GPIO_SPEED_FREQ_LOW
};

typedef struct {
    GPIO_InitTypeDef pin;
    GPIO_TypeDef *port;
    int delay;
    uint8_t counter;
} mygpio_pin;

mygpio_pin pinB0;
mygpio_pin pinB7;

int Init_Thread(void) {
    __HAL_RCC_GPIOB_CLK_ENABLE();

    pinB0.pin = led_ld1;
    pinB0.port = GPIOB;
    pinB0.delay = 15;
    pinB0.counter = 1;
    tid_Thread = osThreadNew(Thread, (void *)&pinB0, NULL);
    if (tid_Thread == NULL) return -1;

    pinB7.pin = led_ld2;
    pinB7.port = GPIOB;
    pinB7.delay = 10;
    pinB7.counter = 0;
    tid_Thread = osThreadNew(Thread, (void *)&pinB7, NULL);
```

```
    if (tid_Thread == NULL) return -1;

    return 0;
}

void Thread(void *argument) {
    mygpio_pin *gpio = (mygpio_pin *)argument;
    HAL_GPIO_Init(gpio->port, &(gpio->pin));
    while (1) {
        gpio->counter++;
        HAL_GPIO_TogglePin(gpio->port, gpio->pin.Pin);
        osDelay(gpio->delay);
    }
}
```

3.7. Dependencias software

- Librería HAL de STM32.
- CMSIS RTOS v2.

3.8. Preguntas y respuestas sobre ejemplothreads

Esta sección contiene una serie de preguntas con sus respectivas respuestas sobre el funcionamiento del código que utiliza CMSIS RTOS v2 para controlar LEDs en una placa STM32.

3.8.1. ¿Qué función hace este código?

Este código crea dos hilos (threads) que controlan dos LEDs conectados a los pines PB0 y PB7 de una placa STM32F4. Cada hilo alterna el estado del LED (encendido/apagado) con una frecuencia determinada utilizando funciones del sistema operativo en tiempo real CMSIS RTOS v2. Es importante entender que el mismo código (función Thread) es ejecutado por dos hilos diferentes, cada uno con sus propios parámetros, que se reciben en el argumento de la función. Es de tipo `void` para poder pasar cualquier tipo de estructura como argumento. Dentro del código del Thread se realiza un casting al tipo de estructura que se utiliza en el ejemplo

3.8.2. ¿Qué función tiene *mygpio_pin*?

Es una estructura de datos que encapsula la información necesaria para controlar un pin GPIO en este ejemplo:

- `pin`: configuración del pin (tipo, velocidad, modo).
- `port`: puerto GPIO al que pertenece el pin (por ejemplo, GPIOB).
- `delay`: retardo en ms entre cada cambio de estado (toggle).
- `counter`: variable auxiliar que cuenta la cantidad de veces que se ha realizado el toggle.

3.8.3. ¿Cómo se inicializan los hilos?

La función `Init_Thread()` habilita el reloj del puerto GPIOB, configura los parámetros de cada LED y crea dos hilos con `osThreadNew()`, pasando como argumento la estructura `mygpio_pin` correspondiente a cada LED.

3.8.4. ¿Qué función tiene `Thread()`?

La función `Thread(void *argument)` es ejecutada por cada hilo. Dentro de ella:

1. Se inicializa el pin GPIO usando `HAL_GPIO_Init`.
2. Se entra en un bucle infinito donde: - Se alterna el valor de `counter`. - Se cambia el estado del LED con `HAL_GPIO_TogglePin`. - Se espera el tiempo definido en `delay` usando `osDelay`.

3.8.5. ¿Se ejecutan los hilos al mismo tiempo?

CMSIS RTOS v2 permite la ejecución concurrente, que no simultanea, de múltiples hilos. El scheduler del sistema operativo se encarga de asignar tiempo de CPU a cada hilo según su estado y prioridad.

3.8.6. ¿Qué función tiene `osDelay()`

Es una función del RTOS que suspende la ejecución del hilo actual durante un número determinado de ticks. Esto permite que otros hilos se ejecuten mientras tanto. `osDelay` tiene como parametro el número de ticks que la tarea estará bloqueada. El número de ticks por segundo se define en el archivo `RTX_Config.h` (parámetro `Kernel Tick Frequency [Hz]`). En este ejemplo se ha configurado a 1000, por lo que un tick equivale a 1 ms.

3.8.7. ¿Qué pasa si `osThreadNew()` retorna NULL?

Significa que no se pudo crear el hilo. En ese caso, la función `Init_Thread()` devuelve -1 como señal de error.

3.8.8. ¿Qué includes se utilizan?

- `cmsis_os2.h`: para funciones del sistema operativo en tiempo real.
- `stm32f4xx_hal.h`: para funciones de acceso a hardware (HAL).
- `stdlib.h`: para funciones estándar de C.

3.8.9. ¿Cuanto vale el valor del tick es esta aplicación?

El fichero de configuración del sistema operativo tal y como indica la figura tiene configurado un tick de 1ms.

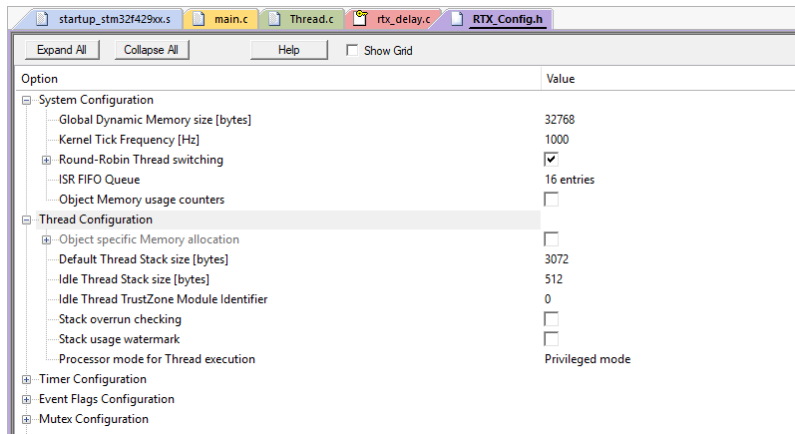


Fig. 3.1 Configuración del sistema operativo.

3.8.10. ¿Que es el thread Idle? ¿Qué tamaño de stack tiene? ¿Y otro thread? ¿Que tamaño de stack usa?

El thread idle esta definido en el fichero RTX_Config.c y es un thread que se ejecuta cuando el sistema operativo no tiene ninguna otro thread que ejecutar. Tiene un tamaño de `stack` de 512 bytes. Cualquier otro thread se configura para tener un tamaño de stack de 3072 bytes (3KBytes). Una reflexión interesante es cuantos threads se pueden crear en una aplicación.

4. Uso básico de threads y colas en CMSIS RTOS v2

Este documento describe el funcionamiento de un programa en C que utiliza CMSIS RTOS v2 y la biblioteca HAL de STM32 para controlar dos LEDs mediante hilos concurrentes que se comunican con colas.

4.1. Descripción General

El programa crea dos hilos, denominados `Producer` y `Consumer`. El hilo `Producer` se encarga de introducir datos en la cola que tiene el identificador `id_MsgQueue`. El numero total de mensajes que se introducen en la cola en cada iteración del bucle `while` es 32 con un tiempo de retardo entre operaciones de escritura que es variable. El `Consumer` se encarga de extraer los datos de cola y de actuar sobre los leds en función del valor leído de la cola.

4.2. Estructuras de Datos

Se define una estructura llamada `mygpio_pin` que encapsula toda la información necesaria para controlar un LED:

- `GPIO_InitTypeDef pin`: configuración del pin (modo, velocidad, tipo de salida).
- `GPIO_TypeDef *port`: puerto GPIO al que pertenece el pin.

Esta estructura permite pasar todos los parámetros necesarios a la función del hilo de forma organizada.

4.3. Creación Hilos

La función `Init_Thread` realiza las siguientes operaciones:

1. Crea una cola con `osMessageQueueNew` para almacenar hasta 16 mensajes cada uno de tamaño un `uint8_t`, es decir, un solo byte.
2. Crea un hilo `Consumer` con `osThreadNew`, ejecutando la función `Consumer`.
3. Crea un hilo `Producer` con `osThreadNew`, ejecutando la función `Producer`.

4.4. Producer

La función `Producer(void *argument)` realiza las siguientes operaciones: 1. De manera continua en un bucle infinito inserta en cola en cada iteración del bucle while 32 mensajes de 1 byte con el valor de la variable index. 2. Los mensajes se introducen con un retardo que varia desde 100ms hasta 400ms 3. La función `osMessageQueuePut` introduce los mensajes con timeout 0, lo cual implica que si no hay sitio en la cola el mensaje no se podrá guardar.

4.5. Consumer

La función `Consumer(void *argument)` realiza las siguientes operaciones: 1. De manera continua en un bucle infinito extrae de la cola los mensajes introducidos por el hilo `Producer`. 2. Si se saca un valor de la cola se procede a encender o apagar los leds en función del valor leído. 3. La variable `errors_or_timeout` cuenta el número de veces que no se ha podido leer un mensaje de la cola, ya sea por timeout o porque la cola está vacía.

4.6. HAL-CMSIS RTOS

- **HAL (Hardware Abstraction Layer):** se utiliza para configurar e inicializar los pines GPIO de forma sencilla y portable.
- **CMSIS RTOS v2:** proporciona las funciones para crear y gestionar hilos, como `osThreadNew` y `osDelay`, y las funciones para gestionar las colas.

4.7. Fuente

```

#include "cmsis_os2.h"           // CMSIS RTOS header
file
#include "stm32f4xx_hal.h"
#include <string.h>
#include <stdlib.h>

osThreadId_t tid_Thread;        // thread id
osMessageQueueId_t id_MsgQueue;
int Init_Thread (void);
void Producer (void *argument); // thread function
producing data
void Consumer (void *argument); // thread function
consuming data
int qsize=0;
uint16_t h=0;
uint8_t i=0;

typedef struct {
    GPIO_InitTypeDef pin;
    GPIO_TypeDef *port;
} mygpio_pin;

mygpio_pin pinB0;
mygpio_pin pinB7;

int Init_Thread (void) {

    id_MsgQueue = osMessageQueueNew(16, sizeof(uint8_t), NULL);

    tid_Thread = osThreadNew(Producer, NULL, NULL);
    if (tid_Thread == NULL) {
        return(-1);
    }

    tid_Thread = osThreadNew(Consumer, NULL, NULL);
    if (tid_Thread == NULL) {
        return(-1);
    }

    return(0);
}

void Producer (void *argument) {

```



```

        uint8_t index=0;
        osStatus_t status;
while (1) {
            for( h=1; h<5; h++){
                for( i=0; i< 8; i++){

status=osMessageQueuePut(id_MsgQueue, &index, 0U, 0U);
                    index++;
                    osDelay(h*100);

                }

            }

}
void Consumer (void *argument) {
    uint8_t val=0;
    osStatus_t status;
    int errors_or_timeouts=0;
    GPIO_InitTypeDef led_ld1 = {
        .Pin = GPIO_PIN_0,
        .Mode = GPIO_MODE_OUTPUT_PP,
        .Pull = GPIO_NOPULL,
        .Speed = GPIO_SPEED_FREQ_LOW
    };
    GPIO_InitTypeDef led_ld2 = {
        .Pin = GPIO_PIN_7,
        .Mode = GPIO_MODE_OUTPUT_PP,
        .Pull = GPIO_NOPULL,
        .Speed = GPIO_SPEED_FREQ_LOW
    };
    __HAL_RCC_GPIOB_CLK_ENABLE();

    HAL_GPIO_Init(GPIOB, &led_ld1);

    HAL_GPIO_Init(GPIOB, &led_ld2);

while (1) {
    qsize=osMessageQueueGetCount (id_MsgQueue);
    status = osMessageQueueGet(id_MsgQueue, &val, NULL,
10U); // wait for message
        if (status == osOK){
            HAL_GPIO_WritePin(GPIOB,led_ld1.Pin,
(GPIO_PinState) val&0x01);
            HAL_GPIO_WritePin(GPIOB,led_ld2.Pin,
(GPIO_PinState)(val&0x02)>>1);

        }
        else {
            errors_or_timeouts++;
        }
        osDelay(250); //This delay is to simulate an
operation that needs 101ms to complete

```

```
}  
}
```

4.8. Dependencias de software

- Librería HAL de STM32.
- CMSIS RTOS v2.

4.9. Preguntas y respuestas sobre ejemplthreads-queues

Esta sección contiene una serie de preguntas con sus respectivas respuestas sobre el funcionamiento del código que utiliza CMSIS RTOS v2 para controlar LEDs en una placa STM32.

4.9.1. ¿Cuál es el propósito de la cola de mensajes *id_MsgQueue* en esta aplicación?

La cola de mensajes *id_MsgQueue* actúa como un canal de comunicación y sincronización entre los hilos *Producer* y *Consumer*. Permite que el hilo productor envíe datos (índices) al consumidor de forma segura y sincronizada. Al definir una cola con capacidad para 16 elementos de tipo *uint8_t*, se establece un buffer temporal que desacopla la producción y el consumo de datos.

4.9.2. ¿Qué función cumple el bucle anidado en el hilo *Producer*?

El bucle anidado en *Producer* genera una secuencia de valores que se colocan en la cola de mensajes. El bucle externo recorre *h* de 1 a 4, y el interno recorre *i* de 0 a 7. En cada iteración, se coloca un valor en la cola (*index*) y se incrementa. El retardo *osDelay(h*100)* introduce una variabilidad en el tiempo entre envíos, oscilando entre 100 ms y 400 ms. Esto simula diferentes tasas de producción de datos.

4.9.3. ¿Cuanto tiempo tarda en llenarse la cola de mensajes *id_MsgQueue*?

En la cola se introducen 32 mensajes en cada ciclo completo de los bucles anidados (8 mensajes por cada uno de los 4 valores de *h*) pero el Thread Consumer extrae mensajes cada 250ms en el caso de que existan. Por tanto la cola nunca llega a llenarse. Intente calcular cual sería el número máximo de mensajes que se pueden acumular en la cola.

4.9.4. ¿cuanto vale la variable `errors_or_timeouts` despues de 1 minuto de ejecución del código?

Vale 0 porque no se produce dicha condición nunca.

Note

Challenge: Modifique el código del hilo `Producer` para que la variable `errors_or_timeouts` no valga cero.

5. Uso básico de threads y flags en CMSIS RTOS v2

Este documento describe el funcionamiento de un programa en C que utiliza CMSIS RTOS v2 y la biblioteca HAL de STM32 para controlar dos LEDs mediante hilos concurrentes que se sincronizan con flags.

5.1. Descripción

El programa crea dos hilos, denominados `Producer` y `Consumer`. El hilo `Producer` es un bucle infinito que se encarga de activar flags para el thread `Consumer`. Este en función de los flags activados ejecuta unas acciones u otras.

Se define una estructura llamada `mygpio_pin` que encapsula toda la información necesaria para controlar un LED:

- `GPIO_InitTypeDef pin`: configuración del pin (modo, velocidad, tipo de salida).
- `GPIO_TypeDef *port`: puerto GPIO al que pertenece el pin.

Esta estructura permite pasar todos los parámetros necesarios a la función del hilo de forma organizada.

La función `Init_Thread` realiza las siguientes operaciones:

1. Crea un hilo `Consumer` con `osThreadNew`, ejecutando la función `Consumer`.
3. Crea un hilo `Producer` con `osThreadNew`, ejecutando la función `Producer`.

5.2. Función del hilo Producer

La función `Producer(void *argument)` realiza las siguientes operaciones: 1. De manera continua en un bucle infinito señala flags en el thread `Consumer`. 2. Los flags activados son el 0x0001 y en 0x0002.

5.3. Función del hilo Consumer

La función `Consumer(void *argument)` realiza las siguientes operaciones: 1. Inicializa dos pines del GPIO en el puerto B. 2. Espera de manera infinita (`osWaitForever`) a que cualquiera (`osFlagsWaitAny`) de los flags (`0` o `1`, en hexadecimal `0x03`) se activen. 3. Si el flag activado es el `0` se hace un toggle en el pin 0 del GPIOB. Si el activado es el `1` se hace el toggle en el pin 7 del GPIOB. 4. Si se produce otra condición se incrementa la variable `errors`.

5.4. HAL y CMSIS RTOS

- **HAL (Hardware Abstraction Layer):** se utiliza para configurar e inicializar los pines GPIO de forma sencilla y portable.
- **CMSIS RTOS v2:** proporciona las funciones para crear y gestionar hilos, como `osThreadNew` y `osDelay`, y la funciones de gestion de los flags.

5.5. Código aplicación

```

#include "cmsis_os2.h"           // CMSIS RTOS header
file
#include "stm32f4xx_hal.h"
#include <string.h>
#include <stdlib.h>

osThreadId_t tid_Thread_producer;           // thread id
osThreadId_t tid_Thread_consumer;
int Init_Thread (void);
void Producer (void *argument);             // thread function
producing data
void Consumer (void *argument);             // thread function
consuming data
int qsize=0;
uint8_t a=0;
uint8_t b=0;

typedef struct {
    GPIO_InitTypeDef pin;
    GPIO_TypeDef *port;
} mygpio_pin;

mygpio_pin pinB0;
mygpio_pin pinB7;

int Init_Thread (void) {

tid_Thread_producer = osThreadNew(Producer, NULL, NULL);
if (tid_Thread_producer == NULL) {
    return(-1);
}

    tid_Thread_consumer = osThreadNew(Consumer, NULL, NULL);
if (tid_Thread_consumer == NULL) {
    return(-1);
}

return(0);
}

void Producer (void *argument) {

```

```

        uint32_t status;
while (1) {

    status= osThreadFlagsSet(tid_Thread_consumer,0x0001);
    osDelay(1000);
    status= osThreadFlagsSet(tid_Thread_consumer,0x0002);
    osDelay(1000);

}
}
void Consumer (void *argument) {
    uint8_t val=0;
    uint32_t status;
    int errors=0;
    GPIO_InitTypeDef led_ld1 = {
        .Pin = GPIO_PIN_0,
        .Mode = GPIO_MODE_OUTPUT_PP,
        .Pull = GPIO_NOPULL,
        .Speed = GPIO_SPEED_FREQ_LOW
    };
    GPIO_InitTypeDef led_ld2 = {
        .Pin = GPIO_PIN_7,
        .Mode = GPIO_MODE_OUTPUT_PP,
        .Pull = GPIO_NOPULL,
        .Speed = GPIO_SPEED_FREQ_LOW
    };
    __HAL_RCC_GPIOB_CLK_ENABLE();

    HAL_GPIO_Init(GPIOB, &led_ld1);

    HAL_GPIO_Init(GPIOB, &led_ld2);

while (1) {
    status=osThreadFlagsWait(0x3,osFlagsWaitAny,osWaitForever);
    switch (status){
        case 1:

            HAL_GPIO_TogglePin(GPIOB,led_ld1.Pin);
                                a=!a;
                                break;

        case 2:

            HAL_GPIO_TogglePin(GPIOB,led_ld2.Pin);
                                b=!b;
                                break;

        default:errors++;
                                break;
    }
}

```

```
}      }
```

5.6. Dependencias de software del ejemplo

- Librería HAL de STM32.
- CMSIS RTOS v2.

5.7. Preguntas y respuestas sobre ejemplothreads-flags

Esta sección contiene una serie de preguntas con sus respectivas respuestas sobre el funcionamiento del código que utiliza CMSIS RTOS v2 para controlar LEDs en una placa STM32.

5.7.1. Se modifica el código del Producer para que envíe ambas señales (0x0001 y 0x0002) de forma casi simultánea, seguido de un delay de 1 segundo:

```
1 void Producer (void *argument) {  
2     uint32_t status;  
3     while (1) {  
4         status = osThreadFlagsSet(tid_Thread_consumer,  
0x0001);  
5         status = osThreadFlagsSet(tid_Thread_consumer,  
0x0002);  
6         osDelay(1000);  
7     }  
8 }
```

Analice el comportamiento resultante del sistema y responda:

1. ¿Qué valor tendría la variable status en el Consumer después de osThreadFlagsWait?
2. ¿Cómo afecta esta modificación al parpadeo de los LEDs?

1. Valor de status: La variable status en el Consumer tendría el valor 0x0003 (0x0001 | 0x0002), ya que los flags se acumulan en el sistema CMSIS-RTOS cuando se envían antes de que el thread destino los procese.
2. Efecto en los LEDs: Los LEDs dejarían de parpadear por completo. El switch statement en el Consumer solo maneja explícitamente los casos 1 (0x0001) y 2 (0x0002). Al recibir el valor combinado 3, la ejecución cae en el caso default, donde solo se incrementa la variable errors sin ejecutar ninguna operación de toggle en los GPIOs.

6. Uso básico de threads y software timers en CMSIS RTOS v2

Esta sección describe el funcionamiento de un programa en C que utiliza CMSIS RTOS v2 y la biblioteca HAL de STM32 para controlar dos LEDs mediante un hilo y timers software.

6.1. Descripción General

El programa crea un único hilo denominado `Timers`. Este hilo se encarga de configurar los pines B0 y B7 como salida para excitar los LEDs LD1 y LD2. Además crea un timer one-shot y otro periódico. El timer one-shot se inicia para que al cabo de 10 segundos se active y en su callback se encienda el led LD1 y se inicie el timer periódico. El timer periódico hace que el led LD2 parpadee cada 500ms.

Se define una estructura llamada `mygpio_pin` que encapsula toda la información necesaria para controlar un LED:

- `GPIO_InitTypeDef pin`: configuración del pin (modo, velocidad, tipo de salida).
- `GPIO_TypeDef *port`: puerto GPIO al que pertenece el pin.

La función `Init_Thread` realiza las siguientes operaciones:

1. Crea un hilo `Timers` con `osThreadNew`.

6.2. Función del Hilo `Timers`

La función `Timers(void *arg)` realiza las siguientes operaciones:

1. Configura los pines GPIO para los LEDs LD1 y LD2.
2. Ejecuta un bucle infinito que en cada iteración espera 1 segundo.

6.3. Uso de HAL y CMSIS RTOS

- **HAL (Hardware Abstraction Layer):** se utiliza para configurar e inicializar los pines GPIO de forma sencilla y portable.
- **CMSIS RTOS v2:** proporciona las funciones para crear y gestionar hilos, como `osThreadNew` y `osDelay`, y funciones específicas para gestionar timers.

6.4. Código específico

```

1 #include "cmsis_os2.h" // CMSIS RTOS
header file
2 #include "stm32f4xx_hal.h"
3 #include <string.h>
4 #include <stdlib.h>
5
6
7 void Init_Threads (void);
8 void Timers (void*);
9 GPIO_InitTypeDef led_ld1 = {
10     .Pin = GPIO_PIN_0,
11     .Mode = GPIO_MODE_OUTPUT_PP,
12     .Pull = GPIO_NOPULL,
13     .Speed = GPIO_SPEED_FREQ_LOW
14 };
15 GPIO_InitTypeDef led_ld2 = {
16     .Pin = GPIO_PIN_7,
17     .Mode = GPIO_MODE_OUTPUT_PP,
18     .Pull = GPIO_NOPULL,
19     .Speed = GPIO_SPEED_FREQ_LOW
20 };
21
22
23 typedef struct {
24     GPIO_InitTypeDef pin;
25     GPIO_TypeDef *port;
26 } mygpio_pin;
27
28 mygpio_pin pinB0;
29 mygpio_pin pinB7;
30 void Timer1_Callback_1(void *arg);
31 void Timer1_Callback_2(void *arg);
32 osTimerId_t timsoft2 ;
33 void Init_Threads(void){
34     osThreadId_t tid_Thread = osThreadNew(Timers, NULL, NULL);
35 }
36 void Timers (void* arg) {
37
38     __HAL_RCC_GPIOB_CLK_ENABLE();
39
40     HAL_GPIO_Init(GPIOB, &led_ld1);
41
42     HAL_GPIO_Init(GPIOB, &led_ld2);
43
44     HAL_GPIO_WritePin(GPIOB, led_ld1.Pin, GPIO_PIN_RESET);

```

```
45     HAL_GPIO_WritePin(GPIOB, led_ld2.Pin, GPIO_PIN_RESET);
46
47     osTimerId_t timsoft1 = osTimerNew(Timer1_Callback_1,
osTimerOnce, NULL, NULL);
48
49     osTimerStart(timsoft1, 10000);
50     timsoft2 = osTimerNew(Timer1_Callback_2, osTimerPeriodic,
NULL, NULL);
51
52
53 while(1){
54     osDelay(1000);
55 }
56 }
57 void Timer1_Callback_1(void *arg){
58
59     HAL_GPIO_TogglePin(GPIOB, led_ld1.Pin);
60     osTimerStart(timsoft2, 500);
61
62 }
63
64 void Timer1_Callback_2(void *arg){
65
66     HAL_GPIO_TogglePin(GPIOB, led_ld2.Pin);
67
68 }
```

6.5. Dependencia

- Librería HAL de STM32.
- CMSIS RTOS v2.

6.6. Preguntas y respuestas sobre ejemplothreads-timers

Esta sección contiene una serie de preguntas con sus respectivas respuestas sobre el funcionamiento del código que utiliza CMSIS RTOS v2 para controlar LEDs en una placa STM32.

6.6.1. ¿Cual es la diferencia fundamental entre un timer periódico otro one-shot?

El timer one-shot dispara la función de callback una sola vez. Es importante indicar que el tiempo empieza a contar desde que el timer es arrancado. Un timer periódico por contra ejecuta la función de callback multiples veces. Es importante hacer notar que la función de arrancar un timer no se puede llamar desde una rutina de atención a la interrupción. La función de arrancar un timer se puede llamar de manera reiterada reiniciando la cuenta de tiempo del mismo.

6.6.2. Los ficheros RTX_config.h y RTX_config.c son generados automáticamente por el entorno de desarrollo. ¿Se pueden modificar?

Sí, se pueden modificar. Estos ficheros contienen configuraciones específicas del sistema operativo en tiempo real (RTOS) RTX, como el número máximo de hilos, la prioridad de los hilos, el tamaño de la pila, entre otros parámetros. Modificar estos archivos permite ajustar el comportamiento del RTOS según las necesidades específicas de la aplicación.

6.6.3. Si se fija un punto de ruptura en la línea 47, ¿qué se espera ver en el `Watch Windows->RTX RTOS`?

1. El hilo en estado running. Además no es el único hilo porque aparece el hilo `osRtxIdleThread` y `osRtxTimerThread`.
2. Se visualiza una cola que es utiliza por el sistema operativo para gestionar eventos internos.

Note

Challenge: Investigue el mecanismo para poder poner su propio código en el thread `osRtxIdleThread`.

Warning

No utilice las funciones de manejo de timers software desde rutinas de atención a la interrupción.

7. Requisitos previos

7.1. Software que se instalará

Software	Versión	Propósito
Visual Studio Code	Última estable	Editor e IDE principal
ARM Keil Studio Pack	MDK v6 (última)	Extensión de VS Code para desarrollo embebido ARM
CMSIS-Toolbox	Última (vía vcpkg)	Herramientas de línea de comandos: <code>cbuild</code> , <code>cpackget</code> , <code>csolution</code>
Arm GNU Toolchain (GCC)	13.x o superior	Compilador <code>arm-none-eabi-gcc</code>
CMake	3.25 o superior	Sistema de build
Ninja	1.10 o superior	Generador de build rápido
Paquete Keil::STM32F4xx_DFP	2.17.x o superior	Soporte de dispositivo para STM32F4
Paquete ARM::CMSIS	6.x	Núcleo CMSIS, headers Cortex-M

7.2. Hardware necesario

- Ordenador con Windows 10/11 (64 bits).
- (Opcional) Placa STM32F429I-Discovery con cable USB para depuración en hardware real.
- (Opcional) Sonda ST-LINK V2/V3.

Note

El compilador y las herramientas se descargan automáticamente por la extensión ARM Keil Studio Pack a través de **vcpkg** la primera vez que

8. Instalación de Visual Studio Code

8.1. Descarga

1. Abre el navegador y ve a <https://code.visualstudio.com/download>
2. Haz clic en el botón **Windows** (descarga el instalador `.exe` de 64 bits).

Tip

Descarga el instalador de **sistema** (*System Installer*) en lugar del de usuario si quieres que VS Code quede disponible para todos los usuarios de la máquina: `VSCodeSetup-x64-<version>.exe`

8.2. Instalación

1. Ejecuta el instalador descargado (doble clic).
2. Acepta el acuerdo de licencia.
3. En la pantalla **Seleccionar tareas adicionales**, marca **todas** las opciones:
 - Agregar acción «Abrir con Code» al menú contextual de archivos.
 - Agregar acción «Abrir con Code» al menú contextual de directorios.
 - Registrar Code como editor predeterminado para los tipos de archivo compatibles.
 - **Agregar a PATH** (*imprescindible*).
4. Haz clic en **Instalar** y espera a que finalice.
5. Haz clic en **Finalizar** con la opción *Iniciar Visual Studio Code* marcada.

8.3. Verificación

Abre un terminal (**Win + R** → **cmd**) y ejecuta:

```
code --version
```

Deberías ver una salida similar a:

```
1.89.1  
b58957e67ee1e712cebf466b995adf4c5307b2bd  
x64
```

Note

Si el comando **code** no se reconoce, cierra y vuelve a abrir el terminal para que recoja la actualización del PATH realizada por el instalador.

8.4. Configuración recomendada del editor

Abre VS Code y accede a la configuración con `Ctrl + ,`. Busca y ajusta los siguientes parámetros:

Parámetro	Valor recomendado
<code>editor.formatOnSave</code>	<code>true</code>
<code>editor.tabSize</code>	<code>4</code>
<code>files.encoding</code>	<code>utf8</code>
<code>files.eol</code>	<code>\\n (LF)</code>
<code>terminal.integrated.defaultProfile.windows</code>	<code>PowerShell</code>

